Koki Woki ChefBot

Technical Reference Report

Objects · Built-in Functions · Architecture

Scala Functional Chatbot — Full Codebase Analysis

Files: ChatBotInteract · ConversationBrain · CookingChatBot · KokiWokiGUI · ChatController · FormattedResponse · LogIn · Preferences · Recipes · Recommendations

Table of Contents

1. Project Overview

Koki Woki ChefBot is a terminal-based and graphical conversational cooking assistant written in Scala 3. It allows users to register, log in, maintain session history, state dietary preferences, and receive personalised recipe suggestions. The system is split across ten source files covering authentication, recipe data, preference persistence, NLP-lite conversation analysis, response formatting, recommendation logic, and a fully-integrated Swing GUI desktop client interface.

File	Package	Primary Role
CookingChatBot.scala	(top-level)	Entry point for CLI, main auth loop, command line chat loop
Recipes.scala	chatBot.data	Recipe case class + 46-recipe knowledge base definitions
ConversationBrain.scala	chatBot.brain	NLP intent/sentiment/topic parser and keyword searcher
Preferences.scala	chatBot.engine	Disk-backed user dietary preference and allergy store
Recommendations.scala	chatBot.recommend	Personalised recipe score recommender with explainability
FormattedResponse.scala	chatBot.response	Stateless card templates, summaries, and help guides formatting
LogIn.scala	chatBot.auth	Registration, user login validation, history text save and restore
ChatBotInteract.scala	chatBot.ui	Central conversational routing ('Chatbot'), state tracking
KokiWokiGUI.scala	chatBot.gui	Desktop GUI window, custom message bubbles, FlatLaf Dark styling
ChatController.scala	chatBot.controller	Mediator bridging GUI window events to core engines

2. Architecture & Package Layout

The codebase follows a layered architecture with strict upward dependency flow. Higher layers depend on lower ones; no circular imports exist. This structure decouples the user-facing presentation layers from core computational systems, allowing high reusability.

Layer 1 — Presentation Layer	KokiWokiGUI (@gui) · CookingChatBot (@main)
Layer 2 — Mediation Layer	ChatController (chatBot.controller)
Layer 3 — Application Router	Chatbot (chatBot.ui)
Layer 4 — Business Services	UserAuth · PreferenceManager · RecommendationEngine
Layer 5 — Core Intel & Data	ConversationBrain · Recipes

The dependency rule: every lower layer is unaware of the layers above it. Recipes has zero imports from other chatBot packages; ConversationBrain depends only on chatBot.data; services depend on data + brain; the mediation controller connects the GUI to the services, and the main entry point starts the application loop.

3. Object Reference — Roles & Responsibilities

3.1 Recipes (chatBot.data — Recipes.scala)

Recipes.scala

Member / Method	Signature / Type	Description
`recipes`	List[Recipe] (val)	All 46 recipes represented as an immutable list
`allCuisines`	List[String] (val)	Distinct cuisine names in lowercase, derived from database
`allTags`	List[String] (val)	Distinct dietary tags in lowercase, derived from database
`allDifficulties`	List[String] (val)	Distinct difficulty levels in lowercase, derived from database

3.2 ConversationBrain (chatBot.brain — ConversationBrain.scala)

ConversationBrain.scala

Member / Method	Signature / Type	Description
`detectIntent(userInput)`	String => String	Parses messages to return Positive_Interest, Negative_Feedback, etc.
`isNegativeSentiment(input)`	String => Boolean	Convenience wrapper testing if intent matches Negative_Feedback
`getUserMood(history)`	List[InteractionEntry] => String	Scores dynamic history count; returns Positive, Negative, Neutral
`extractTopics(history)`	List[IE] => List[String]	Extracts distinct cuisines and ingredient tags from chat logs
`getMostDiscussedTopics(h)`	List[IE] => List[(String,Int)]	Counts keyword mention frequency in session logs, sorted desc
`detectCuisine(input)`	String => Option[String]	Extracts matching known cuisine name from query string
`detectTag(input)`	String => Option[String]	Extracts matching known dietary tag from query string
`detectDifficulty(input)`	String => Option[String]	Extracts matching known difficulty value from query string

3.3 Preferences (chatBot.engine — Preferences.scala)

Preferences.scala

Member / Method	Signature / Type	Description
`storePref(user, pref, val)`	(String,String,String) => Unit	Appends liked/avoided items to user profile text file
`getPrefs(user)`	String => Map[String,List[String]]	Loads preferred/avoided categories from database files on disk
`isAvoided(recipe, prefs)`	(Recipe,Map[String,List[String]]) =>Bool	Tests if a recipe matches any of the user's avoided items

3.4 Recommendations (chatBot.recommend — Recommendations.scala)

Recommendations.scala

Member / Method	Signature / Type	Description
`recommend(u, recs, prefs)`	(String,List[Recipe],Map)=>List[Recipe]	Calculates weights and scores recipes using dynamic preferences
`explainRecommendation(r, pr)`	(Recipe,Map[String,List[String]]) =>Str	Generates plain-text explainability explanations for suggestions

3.5 FormattedResponse (chatBot.response — FormattedResponse.scala)

FormattedResponse.scala

Member / Method	Signature / Type	Description
`formatRecipe(recipe)`	Recipe => String	Renders recipe info cards cleanly in console/text panes
`formatRecipeList(recipes)`	List[Recipe] => String	Aggregates lists of recipes into formatted lists
`showGuide()`	Unit => String	Prints comprehensive manual list of all supported commands

3.6 LogIn (chatBot.auth — LogIn.scala)

LogIn.scala

Member / Method	Signature / Type	Description
`register(user, pass)`	(String, String) => Boolean	Validates and creates credentials database directories on disk
`login(user, pass)`	(String, String) => Boolean	Checks file auth maps to validate password credentials
`saveSession(user, history)`	(String, List[InteractionEntry])=>Unit	Serializes active chat logs directly to local database files

3.7 ChatBotInteract (chatBot.ui — ChatBotInteract.scala)

ChatBotInteract.scala

Member / Method	Signature / Type	Description
`routeInputToHandler(i, u)`	(String, String) => (String, Boolean)	Analyzes user text inputs, updates state, and routes to methods
`currentFlow`	Option[String]	State variable tracking active sub-dialog flow channels
`cookingRecipe`	Option[Recipe]	State cache housing the active target cooking recipe
`cookingStep`	Int	Pointer tracking current recipe step instruction index

3.8 CookingChatBot (top-level — CookingChatBot.scala)

CookingChatBot.scala

Member / Method	Signature / Type	Description
`startChefBot()`	Unit	Loops authentications and loops readLines to drive CLI Shell
`main(args)`	Array[String] => Unit	Standard executable launcher wrapper starting CLI environment

3.9 KokiWokiGUI (chatBot.gui — KokiWokiGUI.scala)

KokiWokiGUI.scala

Member / Method	Signature / Type	Description
`mainFrame`	MainFrame	The primary GUI desktop window container
`chatPanel`	BoxPanel	Flow panel laying out message bubble elements vertically
`addMessage(text, isUser)`	(String, Boolean) => Unit	Thread-safe routine adding styled bubbles and scrolling down

3.10 ChatController (chatBot.controller — ChatController.scala)

ChatController.scala

Member / Method	Signature / Type	Description
`handleInput(text)`	String => String	Relays GUI text events directly to the central Chatbot router
`currentUser`	String	Active session user name string cache

4. Built-in & Standard-Library Functions — Complete Index

The tables below catalogue every Scala built-in collection method, String manipulation function, monadic Option combinator, and external disk os-lib operation leveraged across the chatbot codebase.

4.1 Collection Operations

Function	Used In	Purpose
`.map(f)`	All files	Transforms every element of a List/Seq using function f
`.filter(pred)`	Chatbot, Engine	Keeps only elements satisfying predicate pred
`.flatMap(f)`	Brain, Chatbot	Maps then flattens; used to extract ingredients/tags across recipes
`.find(pred)`	Chatbot, Brain	Returns Option[A] — first element matching predicate or None
`.exists(pred)`	Brain, Chatbot	Returns true if any element satisfies pred
`.forall(pred)`	RecommendEngine	Returns true if all elements satisfy pred (used for None-safe filtering)
`.count(pred)`	Brain (mood)	Counts elements matching pred (counts pos/neg interactions)
`.distinct`	Brain, Chatbot	Removes duplicate elements from a list
`.take(n)`	Brain, Engine	Returns first n elements
`.mkString(sep)`	Formatter, Brain	Joins list elements into a single String with separator
`.isEmpty / .nonEmpty`	All files	Boolean checks on collections and Options
`.size / .length`	Auth, Chatbot	Returns the number of elements in a collection
`.sortBy(f)`	Brain (topics)	Sorts a list by key extracted by f
`.zipWithIndex`	Formatter, Main	Pairs each element with its 0-based index (for step numbering)
`.toList`	Brain, Prefs	Converts Seq / Array / os.ReadLines to immutable List
`.toMap`	PreferenceManager	Converts a List[(K,V)] into a Map[K,V]
`List(...)`	Recipes	Constructs an immutable List literal
`Nil`	Chatbot, Brain	Empty List constant; used to initialise history
`:+`	Chatbot	Appends one element to the end of a List (produces new list)
`++`	Brain	Concatenates two Lists
`Random.shuffle(l)`	RecommendEngine	Returns a randomly reordered copy of the list (scala.util.Random)
`.headOption`	Chatbot	Returns Option of first element; None if list is empty
`.split(regex,-1)`	Brain (loadChat)	Splits string by regex; -1 keeps trailing empty strings
`.flatten`	Chatbot	Flattens a List[Option[A]] or List[List[A]] into List[A]
`.distinct after++`	Brain suggestions	Deduplicates merged autocomplete candidates

4.2 String Operations

Function	Used In	Purpose
`.toLowerCase`	All files	Case-insensitive comparison; normalises user input
`.toUpperCase`	Formatter (guide)	Not used directly but .capitalize is
`.capitalize`	Brain, Chatbot	Uppercases first character of a String
`.trim`	Prefs, Brain, Main	Strips leading/trailing whitespace from strings
`.contains(sub)`	Brain, Chatbot	True if the string contains substring sub
`.startsWith(pre)`	Brain (prefix)	True if string starts with prefix pre (autocomplete)
`.mkString`	Formatter	Joins a collection of strings (see Collections)
`.nonEmpty`	Main, Prefs	True if String length > 0
`.equalsIgnoreCase`	Chatbot, Engine	Case-insensitive string equality check
`.indexOf(ch)`	PreferenceManager	Finds position of ':' separator in preference lines
`.substring(s,e)`	PreferenceManager	Extracts key and value from a preference line
`s"...${expr}"`	All files	String interpolation — embeds Scala expressions in strings
`"""|..."""".stripMargin`	Formatter, Main	Multi-line string with leading '|' stripped
`.split("\\s+")`	Brain (smart search)	Tokenises input on whitespace to get keyword array
`.toInt`	Brain (loadChat)	Parses a numeric String to Int (session sequence numbers)
`.map(_.trim)`	Prefs, Brain	Trims each string in a collection
`.filter(_.contains)`	Prefs	Keeps only lines that contain the ':' separator
`.isWhitespace`	Main (isValid)	Character predicate used to reject blank-space-only inputs

4.3 Option Combinators

Function	Used In	Purpose
`.orElse(opt)`	RecommendEngine	Returns first Some; falls back to second Option if None
`.forall(pred)`	RecommendEngine	True if None OR if Some(v) satisfies pred (safe filter with no pref set)
`.foreach(f)`	Chatbot	Executes side-effect f if Some; no-op if None
`.map(f)`	Brain, Chatbot	Transforms the value inside Some; passes None through
`.isDefined / .isEmpty`	Chatbot	Pattern tests; .isDefined == isSome

4.4 I/O & OS Operations (os-lib)

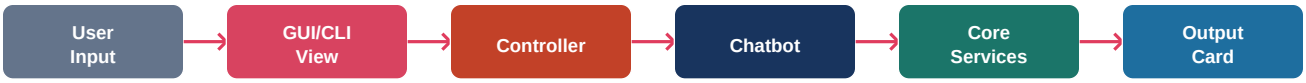
Function	Used In	Purpose
`os.read(path)`	Preferences / LogIn	Reads raw data from dynamic storage files on disk
`os.write(path, data)`	Preferences / LogIn	Writes user accounts and chat sessions to files on disk
`os.exists(path)`	Preferences / LogIn	Checks files presence to validate existing database files

4.5 Control Flow & Pattern Matching

Function	Used In	Purpose
`match { case... }`	All modules	Performs dynamic type-safe pattern matching structures
`guard clauses (if)`	ChatBotInteract / UI	Restricts match options via conditional checks
`Regex patterns`	ChatBotInteract / Brain	Extracts values like preparation times or custom slang greets

5. Data Flow Diagram

The diagram and table below illustrate how a single user message travels through the system from raw stdin input to a formatted response with personalised suggestions.



Step	Action	Component / Detail
1	**User types a message**	stdin via StdIn.readLine() in startChefBot
2	**Input validation**	isValid() and readValid() ensure no blank/whitespace input
3	**generateResponse(input, user)**	Chatbot object — master dispatcher in ChatBotInteract.scala
4	**Sentiment & intent analysis**	ConversationBrain.isNegativeSentiment() + detectIntent()
5	**Context detection**	detectCuisine() / detectTag() / detectDifficulty()
6	**Preference update**	PreferenceManager.storePref() / clearPref() — written to disk
7	**Route to handler**	Pattern match: help / summary / topics / mood / negative / recipe / search
8	**Format response**	ResponseFormatter methods produce the final string
9	**History append**	InteractionEntry added to Chatbot.history in memory
10	**Suggestions**	RecommendationEngine.recommend() reads prefs + contextCuisine
11	**Output to user**	println() in startChefBot prints response and suggestions
12	**Session save (on exit)**	ConversationBrain.saveChat() + UserAuth.saveSession()

6. Key Design Patterns

- **Model-View-Controller (MVC) / Mediator Pattern:** The user presentation layer is separated from database logic. ChatController acts as mediator, eliminating direct cross-reference dependencies and preventing circular imports.
- **Singleton Object Pattern:** Scala's object structures define thread-safe, single shared instances for all manager services (ConversationBrain, RecommendationEngine, PreferenceManager, ResponseFormatter), minimizing memory usage.
- **State Machine Pattern:** Conversational loops and interactive cooking step guides represent dynamic states driven by Option state parameters, allowing robust multi-turn conversations.
- **Template Method / Graphics Painting Hooks:** Overrides paintComponent(g: Graphics2D) inside GUI components to custom draw anti-aliased rounded message bubbles and dynamic background gradient paints.
- **Progressive Relaxation Filter Algorithm:** RecommendationEngine implements progressive relaxation. It filters by exact match first, then drops strict tag constraints sequentially if no results are found, guaranteeing the user always receives relevant recommendations.

End of Report — Koki Woki ChefBot Technical Reference